

Streamlit Documentation: Basic Concepts

Source: <https://docs.streamlit.io/get-started/fundamentals/main-concepts>

Working with Streamlit is simple. First you sprinkle a few Streamlit commands into a normal Python script, then you run it with `streamlit run`:

```
streamlit run your_script.py [-- script args]
```

As soon as you run the script, a local Streamlit server spins up and your app opens in a new tab in your default web browser. The app is your canvas, where you'll draw charts, text, widgets, tables, and more.

`st.text` writes raw text to your app, and `st.line_chart` draws a line chart.

Development Flow

Every time you want to update your app, save the source file. Streamlit detects the change and asks whether you want to rerun your app. Choose "Always rerun" to automatically update your app every time you change its source code. This tight loop between coding and viewing results live is one of the ways Streamlit makes development easier.

Data Flow

Streamlit's architecture allows you to write apps the same way you write plain Python scripts. Streamlit apps have a unique data flow: any time something must be updated on the screen, Streamlit reruns your entire Python script from top to bottom.

This can happen in two situations:

- Whenever you modify your app's source code.
- Whenever a user interacts with widgets in the app — for example, dragging a slider, entering text, or clicking a button.

The `@st.cache_data` decorator allows developers to skip certain costly computations when their apps rerun.

Display and Style Data

Use Magic

Streamlit supports "magic commands" — you don't have to call `st.write()` explicitly. Any time Streamlit sees a variable or a literal value on its own line, it automatically writes that to your app using `st.write()`.

```
import streamlit as st
import pandas as pd
df = pd.DataFrame({
    'first column': [1, 2, 3, 4],
    'second column': [10, 20, 30, 40]
})
df
```

Write a DataFrame

`st.write()` is Streamlit's "Swiss Army knife" — you can pass almost anything to it: text, data, Matplotlib figures, Altair charts, and more.

```
import streamlit as st
import pandas as pd

st.write("Here's our first attempt at using data to create a table:")
st.write(pd.DataFrame({
    'first column': [1, 2, 3, 4],
    'second column': [10, 20, 30, 40]
})))
```

There are also data-specific functions like `st.dataframe()` and `st.table()` for displaying data. Reasons to use a more specific method instead of `st.write()`:

1. You may want to draw data another way — e.g. a static table with `st.table(df)` instead of an interactive one.
2. More specific methods return an object that can be used and modified.
3. A more specific method lets you pass additional arguments to customize its behavior.

```
import streamlit as st
import numpy as np

dataframe = np.random.randn(10, 20)
st.dataframe(dataframe)
```

Draw Charts and Maps

Streamlit supports several popular data charting libraries like Matplotlib, Altair, deck.gl, and more.

```
import streamlit as st
import numpy as np
import pandas as pd

chart_data = pd.DataFrame(
    np.random.randn(20, 3),
    columns=['a', 'b', 'c'])

st.line_chart(chart_data)
```

With `st.map()` you can display data points on a map.

Widgets

Add widgets like `st.slider()`, `st.button()`, or `st.selectbox()`. Treat widgets as variables:

```
import streamlit as st
x = st.slider('x') # this is a widget
st.write(x, 'squared is', x * x)
```

On first run, the app outputs "0 squared is 0". Every time a user interacts with a widget, Streamlit reruns your script from top to bottom, assigning the current state of the widget to your variable in the process.

Widgets can also be accessed by key, if you specify a string to use as the unique key for the widget:

```
import streamlit as st
st.text_input("Your name", key="name")

# You can access the value at any point with:
st.session_state.name
```

Every widget with a key is automatically added to Session State.

Use Checkboxes to Show/Hide Data

```
import streamlit as st
import numpy as np
import pandas as pd

if st.checkbox('Show dataframe'):
    chart_data = pd.DataFrame(
        np.random.randn(20, 3),
        columns=['a', 'b', 'c'])
    chart_data
```

Use a Selectbox for Options

```
import streamlit as st
import pandas as pd

df = pd.DataFrame({
    'first column': [1, 2, 3, 4],
    'second column': [10, 20, 30, 40]
})

option = st.selectbox(
    'Which number do you like best?',
    df['first column'])

'You selected: ', option
```

Layout

Streamlit makes it easy to organize your widgets in a left panel sidebar with `st.sidebar`. Each element passed to `st.sidebar` is pinned to the left, allowing users to focus on the app's content while still having access to UI controls.

```
import streamlit as st

add_selectbox = st.sidebar.selectbox(
    'How would you like to be contacted?',
    ('Email', 'Home phone', 'Mobile phone')
)

add_slider = st.sidebar.slider(
    'Select a range of values',
    0.0, 100.0, (25.0, 75.0)
)
```

Beyond the sidebar, `st.columns` lets you place widgets side-by-side, and `st.expander` lets you conserve space by hiding away large content.

```
import streamlit as st

left_column, right_column = st.columns(2)
left_column.button('Press me!')

with right_column:
    chosen = st.radio(
        'Sorting hat',
        ("Gryffindor", "Ravenclaw", "Hufflepuff", "Slytherin"))
    st.write(f"You are in {chosen} house!")
```

Show Progress

When adding long-running computations to an app, use `st.progress()` to display status in real time.

```
import streamlit as st
import time

'Starting a long computation...'

latest_iteration = st.empty()
bar = st.progress(0)

for i in range(100):
    latest_iteration.text(f'Iteration {i+1}')
    bar.progress(i + 1)
    time.sleep(0.1)

'...and now we\'re done!'
```

Source: Streamlit Documentation, © Snowflake Inc.